2022 IEEE International Conference on Quantum Computing and Engineering (QCE) | 978-1-6654-9113-6/22/$31.00 ©2022 IEEE | DOI: 10.1109/QCE53715.2022.00025

Quantum Representation for Sentiment Classification

Fariska Z. Ruskanda[‡], Muhammad Rifat Abiwardani[‡], Muhammad Akram Al Bari[‡], Kinantan Arya Bagaspati[‡], Rahmat Mulyawan[‡§], Infall Syafalni[‡§] and Harashta Tatimma Larasati[‡¶]
[‡]School of Electrical Engineering and Informatics, Bandung Institute of Technology, Indonesia
[§]University Center of Excellence on Microelectronics, Bandung Institute of Technology, Indonesia
[¶]School of Computer Science and Engineering, Pusan National University, South Korea

***Abstract*—While the application of quantum computing to natural language processing (NLP) has seen substantial work in recent years, not much has been presented on sentiment analysis to systematically identify, extract, quantify, and study affective states and subjective information. This preliminary study investigates how sentiment can be represented correctly and efficiently for quantum natural language processing (QNLP). In particular, we present four possible approaches for representing sentiment words in the quantum case. We propose to utilize two quantum operations (*i.e.*, X gate and Z-X gate), each with two different placements, to represent a negative sentence in a quantum circuit. Subsequently, we train the corresponding state vector using several classical classifiers: SVM, SPSA, and a combination of both, then evaluate the result in terms of accuracy. Experimental results show that the third approach, *i.e.*, adding an X-gate at the end of the sentence outperforms other settings. Additionally, the third approach, along with the combined classifier of SVM and SPSA, reaches up to 81.67% accuracy. The proposed method can be helpful for applications of sentiment analysis in quantum computers.**

***Index Terms*—sentiment classification, SVM, SPSA, quantum representation**

I. Introduction

Recent advances in quantum computing have garnered a significant interest for their applications in various fields, including in natural language processing (NLP). After the pioneering work by Zeng and Coecke [1] that shows the possibility to model natural language in a quantum computer along with its potential advantage, researchers have started to explore quantum NLP (QNLP), *i.e.*, the area that exploits quantum phenomena to design and implement NLP models for performing language tasks on quantum hardware. The works range from general sentence modeling [2], linguistic structure [3] to practical applications (e.g., question answering [4, 5], syntactic classification [6]).

Quantum computing machines possess different characteristics than classical computers, *i.e.*, superposition, entanglement, and interference, which can potentially be employed to solve complex NLP tasks better than their classical counterpart. One of the NLP tasks is sentiment classification, which is a part of sentiment analysis. Sentiment classification determines the sentiment class of an expression or sentence, positive or negative [7]. Given a subjective sentence, for example, "The restaurant was terrible", the sentiment class for the sentence is negative, whereas it is positive for the sentence, "The restaurant served amazing food".

In classical computing, sentiment classification can be performed using traditional machine learning as well as deep learning methods, such as Convolutional Neural Network (CNN) [8], Long-Short Term Memory (LSTM) [9] and its bi-directional variant [10], and transfer learning method [11]. For traditional machine learning, one popular algorithm for sentiment classification is the Support Vector Machine (SVM), which tends to show the best performance among traditional machine learning approaches [12].

In the context of quantum computing, the question is whether the sentiment classification problem can also be solved using the QNLP approach. The primary concern is how to represent subjective sentences on sentiment classification data so that they can be represented and computed correctly and efficiently in a quantum setting. Several studies have proposed the concept of quantum representation, namely by Meichanetzidis *et al.*, [4] for the task question-answering and Lorenz *et al.*, [6] for meaning classification and relative phrase classification task. In [13], the usage of quantum LSTM (Q-LSTM) is proposed for language translation. Additionally, some also use a quantum-like approach to ANN for sentiment analysis, as carried out by Zhang *et al.* in 2019 [14] and 2020 [15]. Nevertheless, to the best of our knowledge, how to specifically formulate and analyze quantum representation for sentiment classification task has not been found in the literature.

In this paper, we examine on how the sentiment of a sentence can be represented in quantum natural language processing (QNLP). In particular, we aim to formulate a suitable and efficient quantum representation for subjective sentences and evaluate it for performing the quantum sentiment classification task. To achieve our goal, we employ a pipeline for experimental QNLP in [16] with several modifications, particularly on the optimization stage, to better suit our case. Furthermore, we experiment with four approaches to represent negative sentiment, which in this preliminary work, is limited to a sentence containing a "not" expression (from now on, referred to as the "Not-Box"). Finally, based on the result from the quantum circuit parameter training, we perform training on the sentiment classification task using classical classifiers: SVM, SPSA, and the combination of both, then discuss the

978-1-6654-9113-6/22/$31.00 ©2022 IEEE
DOI 10.1109/QCE53715.2022.00025

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

result.

The contributions of this paper can be summarized as follows:

1) We formulate a suitable and efficient quantum representation for subjective sentences. We propose to utilize two quantum operations (*i.e.,* X gate and Z-X gate), each with two different placements, to represent a negative sentence in a quantum circuit.
2) We conduct training on sentiment classification tasks with classical classifiers, namely SVM, SPSA, and their combination, and test their performance.

The remainder of this paper is as follows. Section II briefly describes the theoretical backgrounds of sentence representation and the classical approach for sentiment classification. Section III describes our proposed method and presents our preliminary result in the subsequent section. After discussing our result in Section V, Section VI concludes the paper.

II. Related Works

A. Classical Approach for Sentiment Classification

Based on previous research, we would like to review the trend in approaches used for sentiment classification. In their systematic literature review, Nafis *et al.* found that the Support Vector Machine (SVM) is one of the most widely used ML techniques for detecting polarity from text documents [17]. The SVM classifier shows significant performance among other classifiers with the highest accuracy and lowest error rate. Automatic sentiment classification experiments on online Malay-written reviews are carried out using three supervised ML classifiers and seven feature selection techniques [18]. The experimental results showed that feature selection improves Malay sentiment classification. The combination of SVM as feature selection and classifier yielded the highest accuracy of 87%. In addition, compared to Naive Bayes and K-Nearest Neighbor, SVM produces more accurate results for medium and large feature sets. From the previous studies on sentiment classification mentioned above, we concluded that SVM is the most frequent approach used due to its effectiveness. Furthermore SVM is the most frequent classifier tested. It gave competitive classification performances.

In this research, we also use the Simultaneous Perturbation Stochastic Approximation (SPSA) algorithm in tandem with SVM. The essential feature of SPSA—which accounts for its power and relative ease of implementation—is the underlying gradient approximation that requires only two measurements of the objective function regardless of the dimension of the optimization problem. This feature allows for a significant decrease in optimization cost, especially in problems with a large number of variables to be optimized [19]. Gacon *et al.*, have also successfully used SPSA to prepare Hamiltonian ground states and train Variational Quantum Boltzmann Machines (VQBM) [20]. Their experiments showed that using SPSA as the approximation method increased the resulting performance of their VQBM.

B. Methods and Tools for Quantum Natural Language Processing (QNLP)

For the quantum part of this research, Lambeq [16] is used as the library for the Quantum Natural Language Processing (QNLP) part. Lambeq is a high-level Python library for Quantum Natural Language Processing. The open-source toolkit offers a detailed hierarchy of modules and classes implementing all stages of a pipeline for converting sentences to string diagrams, tensor networks, and quantum circuits ready to be used on a quantum computer.

The methods in this paper are based on the work of Coecke *et al.* (2010) [21], Sadrzadeh *et al.* (2013) [22], Meichanetzidis *et al.* (2020) [23], Lorenz *et al.* (2021) [6], and Lambek (2008) [24], which develop a compositional model of meaning using Lambek calculus and pregroup grammar to represent the grammatical syntax of words in sentences and categorical quantum mechanics to translate the pregroups to vector spaces. The details are described in the Appendix.

III. Proposed Method: Quantum Representation for Sentiment Classification

To perform the sentiment classification task with quantum representation, we use a pipeline for experimental QNLP, as used in Kartsaklis [16]. The pipeline generally goes through the following stages: converting sentences into circuits and optimization to get sentiment classification results. The classification process is carried out using a classical classifier: SVM. For this reason, several adjustments were made to the general pipeline, especially at the optimization stage. Hence, our general pipeline stages are (1) generating circuits from sentences, (2) training the parameters of the circuit, (3) extracting the state vectors as sentence embeddings, which are used to (4) train an SVM classifier to predict the sentiment of each sentence, as illustrated in Fig. 1.

This sentiment classification task uses the restaurant sentiment dataset, has binary sentiment (positive/negative), and instantiates sentence type 's' as 1 qubit. The process of conversion from sentence to circuit is adapted from the Zeng-Coecke algorithm [1], Meichanetzidis *et al.* [23], and Lorenz *et al.* [6] with some modifications. The parameter of the circuit training process is carried out in two ways, namely with SPSA and ANN. Our updates for SPSA optimization steps are: convert sentences into trailing cups in the sentence-to-circuit step, apply stemming, and apply Not-Box. Our modifications for ANN optimization include generating symbol pairs and calculating weights to circuit gate parameters. The following is an explanation of the two optimization methods.

A. Sentences-to-Circuit and Circuit-to-Parameter Training

1) SPSA Optimization: **Sentences to circuits:** This step is based on the Quantum Pipeline with Tket available in the Lambeq 0.1.2 documentation [16]. First, sentences from the dataset are converted into DisCoCat diagrams using the DepCCG Parser [25]. Note that the representation of negative sentiment, i.e, "not" word, will be handled separately in later steps. Hence, the current parsing process ignores "not" words

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

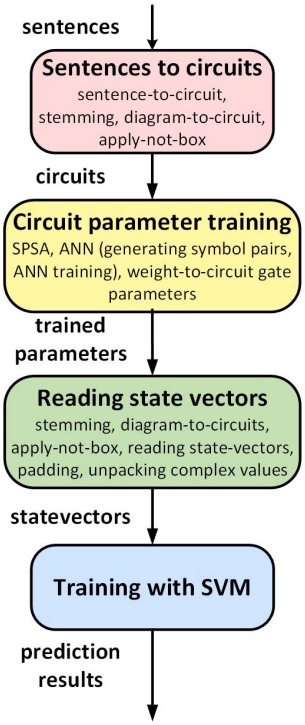


Fig. 1: General pipeline

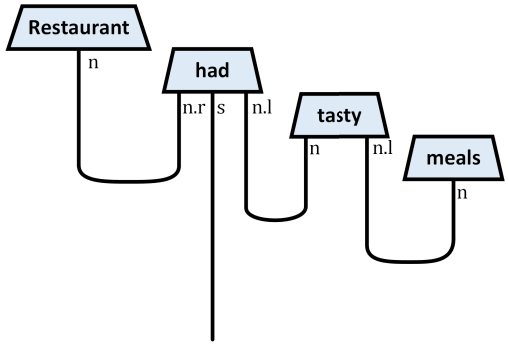


Fig. 2: Before conversion into trailing cups

Algorithm 1: Conversion of sentences to trailing cups

Input: $\mathbb{B}$ consists of K boxes
Output: $\mathbb{D}$ is the new diagram
Dictionary: $\mathbb{W}$: WORDS array, $\mathbb{C}$: CUPS array, N_w: Number of WORDS, N_c: Number of CUPS

begin
  $\mathbb{W} \leftarrow [\emptyset], \mathbb{C} \leftarrow [\emptyset], N_w \leftarrow 0, N_c \leftarrow 0$
  for $k = 0$ *to* $K - 1$ **do**
    if *not domain*$(\mathbb{B}[k])$ **then**
      $\mathbb{W}[N_w] \leftarrow \mathbb{B}[k]$
      $N_w \leftarrow N_w + 1$
    else
      $\mathbb{C}[N_c] \leftarrow \mathbb{B}[k]$
      $N_c \leftarrow N_c + 1$
    end
    for $i = 0$ *to* $N_w - 1$ **do**
      $s \leftarrow$ *length*(*codomain*$(\mathbb{D})$)
      $\mathbb{D} \leftarrow \mathbb{D} \odot \mathbb{I}^s \otimes \mathbb{W}[i]$
    end
    for $i = 0$ *to* $N_c - 1$ **do**
      if $i \neq N_c - 1$ **then**
        $s \leftarrow$ *length*(*codomain*$(\mathbb{D}[:-2])$)
      else
        $s \leftarrow$ *length*(*codomain*$(\mathbb{D}[2:])$)
      end
      $\mathbb{D} \leftarrow \mathbb{D} \odot \mathbb{I}^s \otimes \mathbb{C}[i]$
    end
  end
  return $\mathbb{D}$
end

in a sentence. Subsequently, the diagrams are rewritten using the rewrite rules available in the Lambeq Rewrite package. This package is used to transform string diagrams using certain rewrite rules. String diagrams of some initial form can be transformed by matching its components against a set of transformation rules, then applying the transformation such that the resulting string diagram's strings or boxes are changed accordingly. The main motivation for rewriting is to simplify the diagram, thereby simplifying the resulted training circuit and optimizing the training process. All experiments use the determiner, pre-adverb, and post-adverb rule, while the usage of auxiliary rule varies. The cups in the diagrams are then removed using the bigraph method provided in the Lambeq 0.1.2 documentation.

In order to remove the cups, some diagrams need to be restructured. The restructuring moves all cups below all word boxes. They are ordered such that all the cups on the right of a cup are positioned above it (Fig. 2). Alg. 1 shows pseudocode to perform the conversion, where boxes $\mathbb{B}$ (array) is the input, and the new diagram $\mathbb{D}$ (array) is the output.

Stemming: After the cups are removed, words in the diagram are stemmed using NLTK's PorterStemmer [26].

Diagrams to Circuits: The diagrams are used to build quantum circuits using the IQP Ansatz provided in the Lambeq Ansatz package [16].

Apply Not-Box settings to negative sentences: As the Not-Box settings were removed during the parsing stage, they are applied back into the circuit using one of four representations investigated in this paper: 1) Pauli-X gate on the modified word (Fig. 3(a)); 2) Pauli-Z gate followed by Pauli-X gate on the modified word (Fig. 3(b)); 3) Pauli-X gate on the output qubit (Fig. 3(c)); 4) and Pauli-Z gate followed by Pauli-X gate on the output qubit (Fig. 3(d)). Note that in this paper, the term *output qubit* refers to the qubit representing the resulting type 's' after reduction of grammatical types. In Fig. 3, that is the rightmost string. For other cases where the output qubit

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

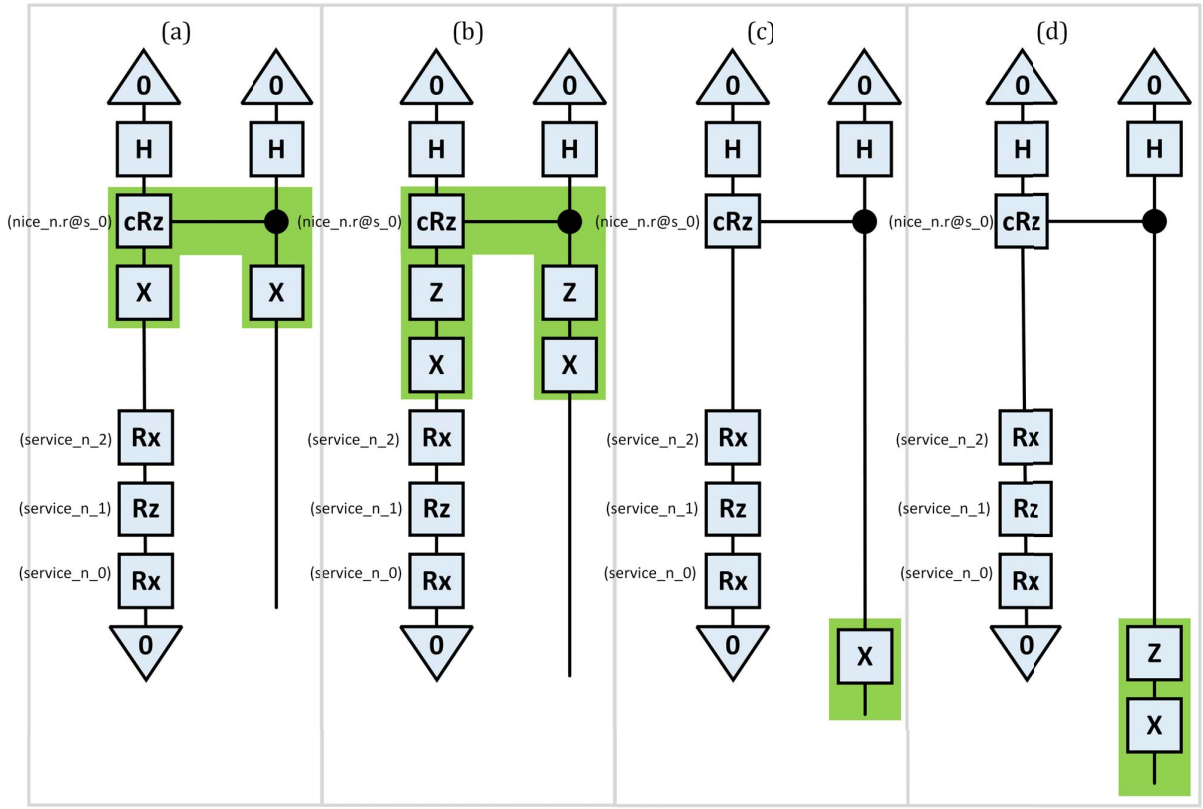


Fig. 3: Not-Box settings: (a) Not1 (X-word), (b) Not2 (Z-X-word), (c) Not3 (X-end), (d) Not4 (Z-X-end)

is located in the middle, the Not-Box may be applied there instead.

The choice to apply the Not-Box on the modified word is inspired by Coecke *et al.* [21], which directly capture the negative meaning of the negated word. The Pauli-X gate is chosen as one of the representations as it flips the probability of measurement on a single qubit, which when applied on the output 's' qubit, corresponds to flipping the sentence's sentiment measured by the circuit. Additionally, the usage of the Pauli-Z gate followed by the Pauli-X gate is inspired by the equation in Lewis [27] which defines the meaning vector of a negated noun "not w" as:

$$[\bar{w}] = I - [w],$$

where $[w]$ is the outer product of $|w\rangle$ with itself and $[\bar{w}]$ is the inversion of $[w]$ (not $[w]$). When $|w\rangle$ spans a single qubit, it can be represented as $\alpha|0\rangle + \beta|1\rangle$, which gives the following derivation:

$$[\bar{w}] = I - |w\rangle\langle w|,$$

where identity matrix I of a single qubit is $I = |0\rangle\langle 0| + |1\rangle\langle 1|$ and we have:

$$|w\rangle\langle w| = \alpha^2|0\rangle\langle 0| + \alpha\beta^*|0\rangle\langle 1| + \alpha^*\beta|1\rangle\langle 0| + \beta^2|1\rangle\langle 1|.$$

By substituting the previous equations, we have:

$$\begin{aligned}[\bar{w}] &= (1-\alpha^2)|0\rangle\langle 0| - \alpha\beta^*|0\rangle\langle 1| - \alpha^*\beta|1\rangle\langle 0| \\ &\quad + (1-\beta^2)|1\rangle\langle 1| \\ &= \beta^2|0\rangle\langle 0| - \alpha\beta^*|0\rangle\langle 1| - \alpha^*\beta|1\rangle\langle 0| + \alpha^2|1\rangle\langle 1|.\end{aligned}$$

Finally, if we find the column vector in $|\bar{w}\rangle$, we have:

$$\begin{aligned}|\bar{w}\rangle &= -\beta^*|0\rangle + \alpha^*|1\rangle \\ &= (-\beta|0\rangle + \alpha|1\rangle)^* \\ &= X(\alpha|0\rangle - \beta|1\rangle) \\ &= XZ(\alpha|0\rangle + \beta|1\rangle).\end{aligned}$$

Therefore, negation can be interpreted as applying a Pauli-X gate (X-end) or a Pauli-Z gate followed by a Pauli-X gate (Z-X-end). In the case of Not-Box using an X-end or a Z-X-end, the measurement of the output will be the same, as follows:

$$P(X|q\rangle = |0\rangle) = P(XZ|q\rangle = |0\rangle) = \beta^*\beta. \quad (1)$$

Intuitively, the reason why the output is similar is because we are currently working on the computational basis state, and the circuit starts with fresh qubits (*i.e.*, initialized at $|0\rangle$). In this case, the Z gate will only flip the phase. Without the presence of a controlled operation in the subsequent step (as shown

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

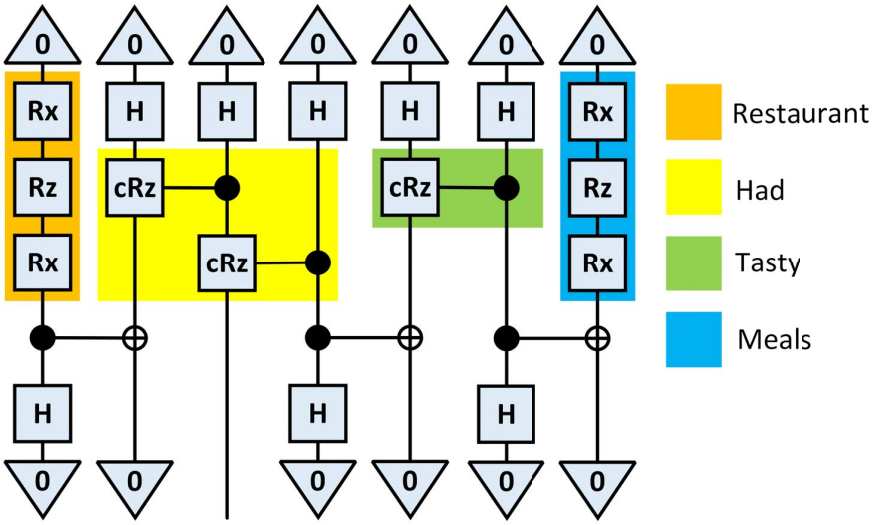


Fig. 4: Example circuit for "the restaurant had tasty meals"

in Figs. 3(b) and (c)), along with the use of computational basis state initialized to $|0\rangle$, the amplitude measurement will be similar to a sole Pauli-X gate. Note that for other cases, the output might yield different result, which we leave for future work.

In addition, a more detailed proof is by evaluating the measurement of the output of an X-end and a Z-X-end. Let the qubit $|q\rangle = \alpha|0\rangle + \beta|1\rangle$. For $X = \{\{0,1\},\{1,0\}\}$, the calculation of the qubit $|q\rangle$ entering the X gate is

$$\begin{aligned} X|q\rangle &= X(\alpha|0\rangle + \beta|1\rangle) \\ &= \beta|0\rangle + \alpha|1\rangle . \end{aligned}$$

The measurement of $X|q\rangle$ is

$$\begin{aligned} P(X|q\rangle = |0\rangle) &= ||\langle 0|X|q\rangle|| \\ &= ||\langle 0|(\beta|0\rangle + \alpha|1\rangle)|| \\ &= ||\langle 0|\beta|0\rangle + \langle 0|\alpha|1\rangle|| \\ &= ||\beta|| = \beta^*\beta. \end{aligned}$$

The same amplitude measurement result is obtained when the Z, then X gates, is applied to qubit $|q\rangle$ as follows:

$$\begin{aligned} XZ|q\rangle &= XZ(\alpha|0\rangle + \beta|1\rangle) \\ &= X(\alpha|0\rangle - \beta|1\rangle) \\ &= -\beta|0\rangle + \alpha|1\rangle . \end{aligned}$$

Finally, the measurement of $XZ|q\rangle$ yields

$$\begin{aligned} P(XZ|q\rangle = |0\rangle) &= ||\langle 0|XZ|q\rangle|| \\ &= ||\langle 0|(-\beta|0\rangle + \alpha|1\rangle)|| \\ &= ||\langle 0| - \beta|0\rangle + \langle 0|\alpha|1\rangle|| \\ &= ||-\beta|| = \beta^*\beta. \end{aligned}$$

In this case, we will have the exact same amplitude measurement results (*i.e.*, measured in computational basis) for measuring the output of X-end or Z-X-end Not-Box setting. Nevertheless, even though we have proven —intuitively and mathematically —that both settings yield the same output in the computational basis state, we still perform the experiment to verify how they behave in the simulation.

In addition, the reason of applying a Not-Box setting at the output is often a logical meaning of a whole sentence is in the reversed meaning after a not word is attached. Meanwhile, in the cases of X-word and Z-X word, the state of qubit $|q\rangle$ is different at the output because of the additional Z gates at before and after the negated word.

Circuit to parameter training: The circuits are compiled into the objective function used for SPSA Optimization [28]. The output qubit of type 's' is compared to the corresponding sentiment label, which is used to calculate the model's cost. This process trains the free variables of the circuits, *i.e.,* the parameters of the gates used in the circuits. After the parameters are optimized, a sentence's sentiment is predicted by instantiating the values of the parameters in its circuit and measuring the output qubit.

2) ANN Optimisation: **Sentences to circuits:** Identical to SPSA optimization, the sentences are first parsed into string diagrams, then rewritten using the Lambeq Rewrite package.

Stemming, Diagrams to Circuits, Apply Not-Box settings to the negative sentences: Similar to the second step to the fourth step of SPSA optimisation method.

Generate symbol pairs: As the parameters of the circuit are not yet instantiated, learning is done using a method adapted from Word2Vec [29]. Instead of word pairs, the method generates symbol pairs. For example, the sentence "the restaurant had tasty meals" would have symbol pairs as the

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

following:

$$n_{restaur_0} \textit{ and } n_{restaur_1},$$
$$n_{restaur_1} \textit{ and } n_{restaur_0},$$
$$n_{restaur_1} \textit{ and } n_{restaur_2},$$
$$n_{restaur_2} \textit{ and } n_{restaur_1},$$
$$n_{restaur_2} \textit{ and } n^r \otimes s \otimes n^l_{had_0},$$
$$n^r \otimes s \otimes n^l_{had_0} \textit{ and } n_{restaur_2}$$

and so on, given a window size of 1 from the circuit in Fig. 4. Pseudocode in generating the list of symbol pairs is shown in Alg. 2, where circuits $\mathbb{U}$ (array) is the input and lists $\mathbb{P}$ (array) is the list of symbol pairs.

Algorithm 2: Generating list of symbol pairs

Input: $\mathbb{U}$ consists of K circuits
Input: N_s is the window size
Output: $\mathbb{P}$ consists of symbol pairs ζ_1 and ζ_2 list
Dictionary: l_1: index of the first symbol, l_2: index of the second symbol, l_3: index of symbol pairs arrays

```
begin
    ℙ ← [∅], l1 ← 0, l2 ← 0, l3 ← 0
    for k = 0 to K − 1 do
        N ← length(𝕌[k])
        for i = 0 to N − 1 do
            if 𝕌[k][i] has a free symbol then
                ζ1 ← 𝕌[k][i]
                l1 ← i
                for j = 0 to N − 1 do
                    if |i − j| ≤ Ns&&i ≠ j then
                        if 𝕌[k][j] has a free symbol then
                            ζ2 ← 𝕌[k][j]
                            l2 ← j
                            ℙ[l3] ← (ζ1, ζ2)
                            l3 ← l3 + 1
                        end
                    end
                end
            end
        end
    end
    return ℤ
end
```

ANN training: An ANN model is built using the symbol pairs generated from the training circuits. Each symbol is given an index number, and the symbol pairs are turned into vectors using one-hot encoding [30]. The ANN model contains an input layer, one hidden layer, and an output layer. The first symbol of each symbol pair is used as an input, with the corresponding pair as the target. This paper uses a hidden layer with two neurons. The weights of the neural network are trained.

Weights to circuit gate parameters: The weight between the i-th input and the first hidden neuron and the weight between the i-th input and the second hidden neuron are used to instantiate the parameters of the circuits. In other words, the i-th symbol is represented by the vector (w_{i0}, w_{i1}) where w_{ij} is the weight between the i-th input neuron and the j-th hidden neuron. An example of the symbol embeddings for "$n^r \otimes s_{fast_0}$", "$n^r \otimes s_{slow_0}$", "$n^r \otimes s_{tasti_0}$", and "$n^r \otimes s_{unappet_0}$" after training is shown in Fig. 5.

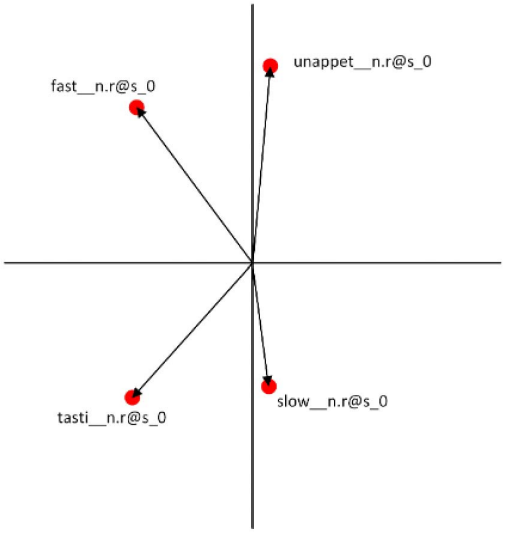


Fig. 5: Example of symbol embedding

This paper proposes to use the angle between the x-axis and the associated vector representation to calculate the value of the i-th circuit parameter. The angle is used because the circuit gates take an angle as its parameter. From the definition of arccosine, given the vector (w_{i0}, w_{i1}):

$$\theta = \arccos \frac{w_{i0}}{\sqrt{w_{i0}^2 + w_{i1}^2}}.$$

As the quantum gates class in the Discopy library define the gate phases as inputs, to get the phase k in $k\pi/2$, these angles are divided by π and multiplied by 2.

B. Reading State Vectors

Stemming and Diagrams to circuits: Training is done on the state vector values of each circuit. State Vectors are read from the boxes of the sentence, taken from the following part of a string diagram (Fig. 6).

That is, the vector representation of a sentence is based on the tensor product of the states of each word box in the sentence. Therefore, the circuits are made from diagrams whose cups were not removed in order to measure the states as in the figure above. Alg. 3 is an example pseudocode to find the cutoff point in each circuit in order to create these prepared states, where the input is $\mathbb{U}$ circuits.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

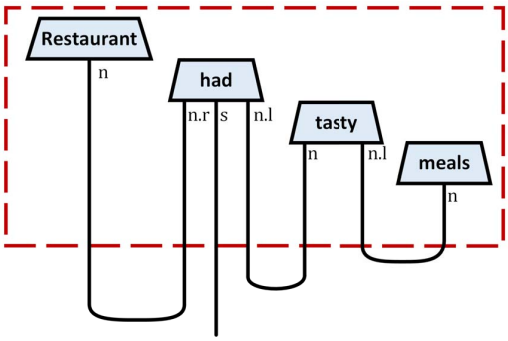


Fig. 6: Diagram boxes after trailing cups

Algorithm 3: Finding cutoff point

Input: $\mathbb{U}$ consists of K circuits
Output: γ is the cutoff point
Dictionary: N: length of the circuit $\mathbb{U}$

```
begin
    γ ← 0, K ← length(U)
    for i = 0 to K − 1 do
        if U[k] has a free symbol then
            γ ← k + 1
        end
    end
    return γ
end
```

Apply Not-Box settings to negative sentences: The Not-Box settings are also applied to sentences containing the word "not", according to the representation chosen.

Read state vectors and add padding: The circuits' parameters are then instantiated, and the state vector values are read. As feature size needs to be uniform among all inputs, all state vectors are padded with the $|0\rangle$ state in order to fill quantum registers of the same size.

Unpacking complex values: For training with classical SVM, as state vectors contain complex values, the values need to be unpacked. In this step, each state vector element is interpreted as two elements in the SVM dataset representing the real part and the imaginary part. For example, a quantum circuit with 7 qubits would have a state vector of length 128, which would be decomposed into 256 real-value elements.

Training with SVM: Finally, these inputs are trained with classical SVM using the RBF kernel [31].

IV. Experimental Results

The experiments in this paper explore and compare three classification methods for identifying the sentiment of a sentence: SPSA classification, SVM classification after learning embeddings with SPSA, and SVM classification after learning embeddings with ANN. These experiments are executed by implementing each step in the general pipeline in Figure 1. The purposes of our experiments are:

1) To study the effect of using Not-Box on each algorithm
2) To study the impact of the rewriter's rule on each algorithm (all or partial)
3) To identify the classification algorithm suitable for the task and data

We built a dataset containing simple subjective sentences for experimental purposes, each consisting of 4-5 words. These sentences were generated from a 29-word vocabulary for the restaurant domain and the patterns, which consist of positive and negative sentences, as shown as follows:

- I like/love/dislike/hate the restaurant
- The restaurant was [adjective]
- The food/meal/service/staff was [adjective]
- The food/meal/service/staff was not [adjective]
- I liked/loved/disliked/hated the [adjective] food/service
- I had [adjective] food/meals/service/staff
- The restaurant had [adjective] food/meals/service/staff
- The restaurant cooked [adjective] food/meals
- The restaurant showed [adjective] service/staff

The dataset consists of 170 training sentences, 50 development sentences, and 60 test sentences. Each sentence is labeled with 0 or 1 denoting negative and positive sentiment respectively.

We used a classical machine to run this experiment, which is mainly needed for SPSA optimization training. The machine environment used is VM type n1-highmem-8, with 8 vCPUs and 52 GB of RAM on Linux OS. In addition, because it is still a preliminary study, benchmark with other approaches is left for our future work. Following the experimental objectives that have been formulated, we carried out two experiments: Not-Box experiments and rewriter rule experiments. The two following subsections are the explanation of each of these experiments. Note that to ensure reproducibility, the same seed was used for each experiment.

A. Not-Box Experiments

Not-Box experiments were conducted to study the effect of using Not-Box on each algorithm. As per the steps in the method in Section III, we use four different Not-Box representations, namely:

1) Not1 (X-word): Pauli-X gate on the modified word;
2) Not2 (Z-X-word): Pauli-Z gate followed by Pauli-X gate on the modified word;
3) Not3 (X-end): Pauli-X gate on the output qubit;
4) Not4 (Z-X-end): Pauli-Z gate followed by Pauli-X gate on the output qubit.

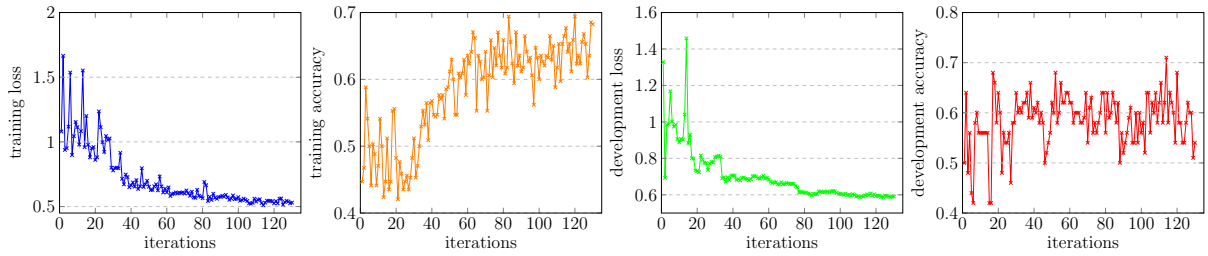
The all-rewrite rule and the bigraph method were used for those four experiments. The complete results of the experiment can be seen in Table I.

B. Rewriter Rule Experiments

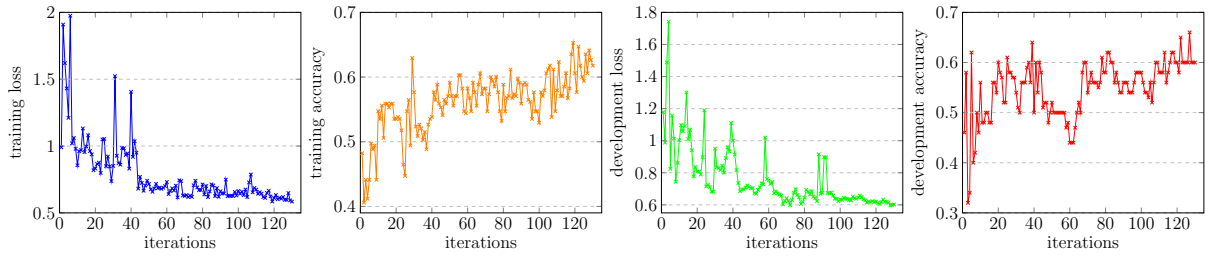
Rule-rewriter experiments were conducted to study the impact of the rewriter's rule on each algorithm,

1) All: all-rewrite rule
2) Partial: partial-rewrite rule, not including auxiliary rule.

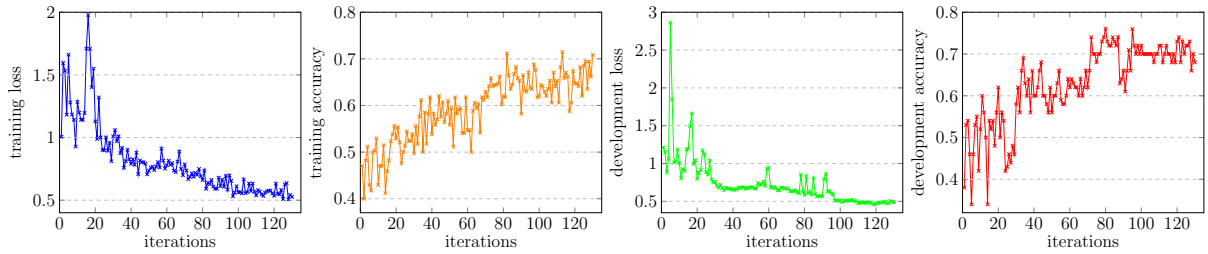
Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.



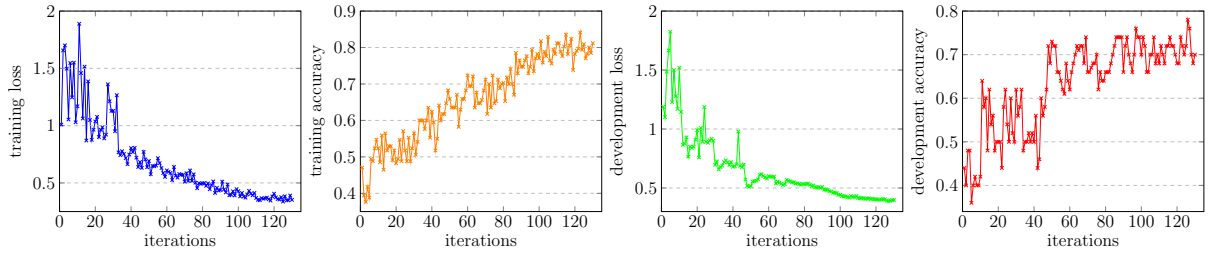
(a) X-end with partial-rewrite rule



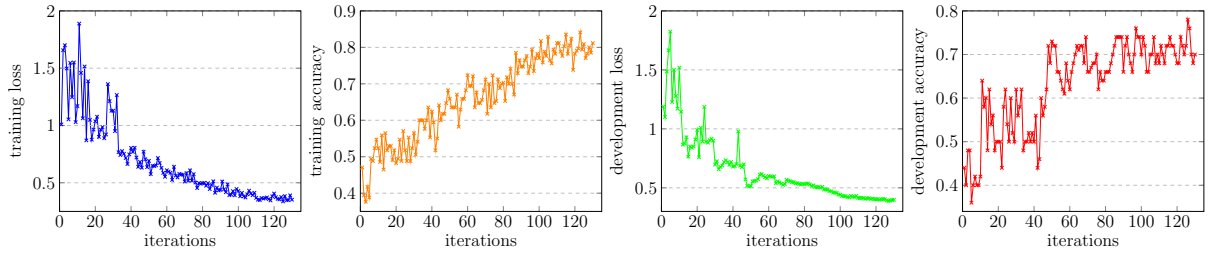
(b) X-word with all-rewrite rule (Not1)



(c) Z-X-word with all-rewrite rule (Not2)



(d) X-end with all-rewrite rule (Not3)



(e) Z-X-end with all-rewrite rule (Not4)

Fig. 7: Results for several settings of Not-Box

In this experiment, we use the best parameter in the previous experiment: the X-end Not-Box. In Figs. 7(a)-(e), the trainings is conducted up to 130 iterations. Moreover, we compare the accuracy between all (Figs. 7(b)-(e)) and partial rewriter's rule (Fig. 7(a)). As shown in Table II, by using all rewriter's rule and the best Not-Box (X-end), the accuracy reaches up to 81.67% for SPSA and SPSA+SVM algorithms.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Not-Box Experimental Results

Representation			Algorithm (Accuracy)			
Rewrite rule	**Bigraph**	**Not-Box**	**SVM**	**SPSA**	**SPSA+SVM**	**Average**
all	on	not1 (X-word)	75.00%	61.67%	73.33%	70.00%
		not2 (Z-X-word)	76.67%	65.00%	78.33%	73.33%
		not3 (X-end)	**78.33%**	**81.67%**	**81.67%**	**80.56%**
		not4 (Z-X-end)	76.67%	**81.67%**	78.33%	78.89%

TABLE II: Rule-Rewriter Experimental Results

Representation			Algorithm (Accuracy)			
Not-Box	**Bigraph**	**Rewrite Rule**	**SVM**	**SPSA**	**SPSA +SVM**	**Average**
not3 (X-end)	on	all	**78.33%**	**81.67%**	**81.67%**	**80.56%**
not3 (X-end)	on	partial	75%	68.33%	66.67%	70%

V. Discussions

Several takeaways from the experimental results in the previous section are as follows:

Not-Box Representation: The most optimal Not-Box representation that we find is using the Pauli-X gate at the end of the sentence (*i.e.*, X-end), with an average accuracy of 80.56%, as shown in Table I, column 7. In the type of sentence with a negation used in our dataset, the impact of the word "not" makes the sentence have the opposite sentiment. For example, the sentence "the food was good" with positive sentiment in the form of "not" in the dataset is "the food was not good" which has a negative sentiment. This characteristic is very close to the nature of the Pauli-X gate in the output qubit.

Meanwhile, in terms of comparison between X-end and Z-X-end (Figs. 7(d) and (e)), the results (training loss, training accuracy, development loss, and development accuracy graphs) are identical. These are inline with the proof of Eq. (1).

In contrast, regarding the comparison between X-word and Z-X-word experiments (Figs. 7(b) and (c)), the resulting graphs differ, reflecting a different optimization path as the post-selection performed on non-output qubits are now affected by the placement of Not-Boxes. From the graphs, we conclude that the application of the Z-X-word Not-Box is more appropriate than the X-word Not-Box for negative sentences.

Classifier: Overall, the classifier that gives the most optimal results for this dataset is SPSA+SVM, with an accuracy score up to 81.67%. The SPSA+SVM outperforms other classifiers possibly because embedding in SPSA is specifically trained for the sentiment analysis task, while ANN in the SVM classifier is trained unsupervised. On the other hand, the X-word and Z-X-word Not-Box representation may have lower accuracy because they affect the word modified by "not" and not the sentence itself. As a result, finding the optimal word gate would be more difficult in terms of minimizing errors on sentiment classification tasks.

Rewrite Rule: The all-rewrite rule gives the best performance with an average accuracy of 80.56% (Table II, column 7). Meanwhile, the partial-rewrite rule is less accurate (70%) for the sentiment sentences, which have positive connotations. The partial-rewrite rule, which does not use the auxiliary rule, makes it necessary to learn auxiliary words such as "was". The dataset does not contain sentences with negative connotations for "was" such as "when I came here two years ago, the food was still good." Using the partial-rewrite rule makes errors when predicting sentiment in sentences without negative connotations. The handling of this contextual semantic element needs to be a concern for further studies.

VI. Conclusions and Future Work

This paper presents a preliminary result of the proposed quantum representation for sentiment classification task. This quantum representation represents subjective sentences, in the form of state vectors, with special handling of negative sentences with the so-called Not-Box operations. The experimental results using a combination of SPSA and SVM classifiers show that this quantum representation is adequate for capturing the semantic side in the form of sentiment with a maximum accuracy of 81.67%.

The success of implementing a quantum representation for the sentiment classification task has potential consequences for future works. The reliability of this quantum representation needs to be benchmarked with the classical approach. Different negative representations can be further investigated, and a more complex form of subjective sentences will need to be explored. The applicability of these quantum representations is essential to be examined in combination with practical Quantum Machine Learning (QML) algorithms and tested on a real quantum machine. Finally, the dataset and sentence size used need to be expanded to handle various subjective sentence representations.

Acknowledgment

This work is supported by the PPMI - Young Researcher Grant 2022 from STEI ITB.

References

[1] W. Zeng and B. Coecke, "Quantum algorithms for compositional natural language processing," *arXiv preprint arXiv:1608.01406*, 2016.

[2] S. Wu, J. Li, P. Zhang, and Y. Zhang, "Natural language processing meets quantum physics: A survey and categorization," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3172–3182.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

[3] N. Wiebe, A. Bocharov, P. Smolensky, M. Troyer, and K. M. Svore, "Quantum language processing," *arXiv preprint arXiv:1902.05162*, 2019.
[4] K. Meichanetzidis, A. Toumi, G. de Felice, and B. Coecke, "Grammar-aware question-answering on quantum computers," *arXiv preprint arXiv:2012.03756*, 2020.
[5] B. Coecke, G. de Felice, K. Meichanetzidis, and A. Toumi, "Foundations for near-term quantum natural language processing," *arXiv preprint arXiv:2012.03755*, 2020.
[6] R. Lorenz, A. Pearson, K. Meichanetzidis, D. Kartsaklis, and B. Coecke, "Qnlp in practice: Running compositional models of meaning on a quantum computer," *arXiv preprint arXiv:2102.12846*, 2021.
[7] B. Liu, *Sentiment analysis: Mining opinions, sentiments, and emotions*. Cambridge university press, 2020.
[8] S. Zhang, X. Xu, Y. Pang, and J. Han, "Multi-layer attention based cnn for target-dependent sentiment classification," *Neural processing letters*, vol. 51, no. 3, pp. 2089–2103, 2020.
[9] Y. Wang, M. Huang, X. Zhu, and L. Zhao, "Attention-based lstm for aspect-level sentiment classification," in *Proceedings of the 2016 conference on empirical methods in natural language processing*, 2016, pp. 606–615.
[10] Z. Hameed and B. Garcia-Zapirain, "Sentiment classification using a single-layered bilstm model," *Ieee Access*, vol. 8, pp. 73 992–74 001, 2020.
[11] S. Dong and C. Liu, "Sentiment classification for financial texts based on deep learning," *Computational Intelligence and Neuroscience*, vol. 2021, 2021.
[12] A. Jain and V. Jain, "Sentiment classification using hybrid feature selection and ensemble classifier," *Journal of Intelligent & Fuzzy Systems*, vol. 42, no. 2, pp. 659–668, 2022.
[13] M. Abbaszade, V. Salari, S. S. Mousavi, M. Zomorodi, and X. Zhou, "Application of quantum natural language processing for language translation," *IEEE Access*, vol. 9, pp. 130 434–130 448, 2021.
[14] Y. Zhang, D. Song, P. Zhang, X. Li, and P. Wang, "A quantum-inspired sentiment representation model for twitter sentiment analysis," *Applied Intelligence*, vol. 49, no. 8, pp. 3093–3108, 2019.
[15] Y. Zhang *et al.*, "A quantum-like multimodal network framework for modeling interaction dynamics in multiparty conversational sentiment analysis," *Information Fusion*, vol. 62, pp. 14–31, 2020.
[16] D. Kartsaklis *et al.*, "Lambeq: An efficient high-level python library for quantum nlp," *arXiv preprint arXiv:2110.04236*, 2021.
[17] N. S. M. Nafis and S. Awang, "An enhanced hybrid feature selection technique using term frequency-inverse document frequency and support vector machine-recursive feature elimination for sentiment classification," *IEEE Access*, vol. 9, pp. 52 177–52 192, 2021.
[18] J.-R. Chang, H.-Y. Liang, L.-S. Chen, and C.-W. Chang, "Novel feature selection approaches for improving the performance of sentiment classification," *Journal of Ambient Intelligence and Humanized Computing*, pp. 1–14, 2020.
[19] A. Liu, X. Deng, Z. Tong, Y. Luo, and B. Liu, "A simultaneous perturbation stochastic approximation enhanced teaching-learning based optimization," in *2016 IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2016, pp. 3186–3192.
[20] A. Boiarov, O. Granichin, and H. Wenguang, "Simultaneous perturbation stochastic approximation for clustering of a gaussian mixture model under unknown but bounded disturbances," in *2017 IEEE Conference on Control Technology and Applications (CCTA)*, IEEE, 2017, pp. 1740–1745.
[21] B. Coecke, M. Sadrzadeh, and S. Clark, "Mathematical foundations for a compositional distributional model of meaning," *arXiv preprint arXiv:1003.4394*, 2010.
[22] M. Sadrzadeh, S. Clark, and B. Coecke, "The frobenius anatomy of word meanings i: Subject and object relative pronouns," *Journal of Logic and Computation*, vol. 23, no. 6, pp. 1293–1317, 2013.
[23] K. Meichanetzidis, S. Gogioso, G. De Felice, N. Chiappori, A. Toumi, and B. Coecke, "Quantum natural language processing on near-term quantum computers," *arXiv preprint arXiv:2005.04147*, 2020.
[24] J. Lambek, *From Word to Sentence: a computational algebraic approach to grammar*. Polimetrica sas, 2008.
[25] R. Yeung and D. Kartsaklis, "A ccg-based version of the discocat framework," *arXiv preprint arXiv:2105.07720*, 2021.
[26] M. F. Porter, "An algorithm for suffix stripping," *Program*, 1980.
[27] M. Lewis, "Towards logical negation for compositional distributional semantics," *arXiv preprint arXiv:2005.04929*, 2020.
[28] J. C. Spall, "An overview of the simultaneous perturbation method for efficient optimization," *Johns Hopkins apl technical digest*, vol. 19, no. 4, pp. 482–492, 1998.
[29] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
[30] D. Harris and S. L. Harris, *Digital design and computer architecture*. Morgan Kaufmann, 2010.
[31] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, no. 3, pp. 273–297, 1995.
[32] J. Lambek, "Type grammar revisited," in *International conference on logical aspects of computational linguistics*, Springer, 1997, pp. 1–27.

APPENDIX: SENTENCE REPRESENTATION USING GRAPHICAL LANGUAGE

A category is formed by the class of objects in the category and the morphisms that relate two objects within the category. Morphisms in a category can be composed, and they all satisfy the associativity property, *i.e.*, for all objects A, B, C, D and

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

morphisms $f : A \to B$, $g : B \to C$, and $h : C \to D$, we have the following:

$$h \odot (g \odot f) = (h \odot g) \odot f.$$

For every object X, there is also the identity morphism $1_X : X \to X$ such that for all objects A, B and the morphism $f : A \to B$, we have the following:

$$1_B \odot f = f = f \odot 1_A.$$

A monoidal category is a category C with a bifunctor $\otimes : C \times C \to C$ called the tensor product or monoidal product. The tensor product is associative, *i.e.*, for all objects A, B, C the following is true:

$$(A \otimes B) \otimes C = A \otimes (B \otimes C).$$

For every object A, there is also the identity functor I such that the following is true:

$$I \otimes A = A = A \otimes I.$$

For all objects A, B, C, D and morphisms $f : A \to C$, $g : B \to D$, the morphism $f \otimes g : A \otimes B \to C \otimes D$ satisfies the following property:

$$(g_1 \otimes g_2) \odot (f_1 \otimes f_2) = (g_1 \odot_1) \otimes (g_2 \odot f_2).$$

A symmetric monoidal category is a monoidal category where the tensor product is symmetric, *i.e.*, for all objects A, B, there is a natural isomorphism η such that $A \otimes B \to B \otimes A$ under η is an isomorphism.

Coecke *et al.* (2010) [21] designs a graphic system for depicting monoidal categories composed of wires and boxes, referred to as string diagrams in this paper. Wires depict the inputs and outputs of a box, where boxes are morphisms. Fig. 8(a)-(c) are several examples of string diagrams for several morphisms where $f : A \to B$ and $g : B \to C$. There are also states and effects, where for an object A, the state ψ maps $\psi : I \to A$ and the effect π maps $\pi : A \to I$ as shown in Fig. 8(d).

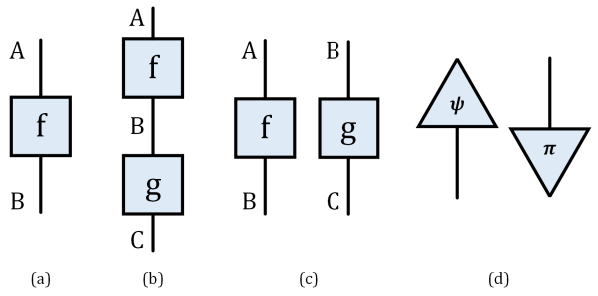


Fig. 8: String diagrams: (a) $f : A \to B$, (b) $f \odot g : A \to C$, (c) $f \otimes g : A \otimes B \to B \otimes C$, (d) $\psi : I \to A$ and $\pi : A \to I$

A compact closed category is a monoidal category where each object A has a left and right adjoint, A^l and A^r respectively, and the unit and co-unit morphisms $\eta^l_A : I \to A \otimes A^l$, $\eta^r_A : I \to A^r \otimes A$, $\epsilon^l_A : A^l \otimes A \to I$, $\epsilon^r_A : A \otimes A^r \to I$. The unit and co-unit morphisms satisfy the yanking equations, *i.e.*, for every object A, we have the following:

$$(1_A \otimes \epsilon^l) \odot (\eta^l \otimes 1_A) = 1_A$$
$$(\epsilon^r \otimes 1_A) \odot (1_A \otimes \eta^r) = 1_A$$
$$(\epsilon^l \otimes 1_{A^l}) \odot (1_{A^l} \otimes \eta^l) = 1_{A^l}$$
$$(1_{A^r} \otimes \epsilon^r) \odot (\eta^r \otimes 1_{A^r}) = 1_{A^r}$$

Coecke *et al.* (2010) [21] draws the yanking equations as a wiring configuration where μ morphisms are drawn as caps and ϵ morphisms are drawn as cups. For example, Fig. 9 is the string diagram for $(1_A \otimes \epsilon^l) \odot (\eta^l \otimes 1_A) = 1_A$.

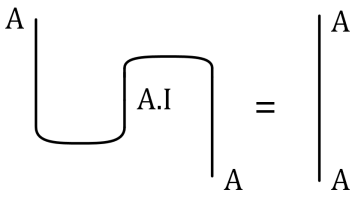


Fig. 9: String diagram for $(1_A \otimes \epsilon^l) \odot (\eta^l \otimes 1_A) = 1_A$

A partially ordered monoid is a partially ordered set $(P, \leq, \cdot, 1)$ where $\cdot$ is the monoid multiplication operator and 1 is the identity element. The monoid multiplication is associative, and for all objects p, q, r in P, where $p \leq q$, we have the following:

$$r \cdot p \leq r \cdot q, p \cdot r \leq q \cdot r.$$

A pregroup, as introduced in Lambek (1999) [32] and explained in Coecke *et al.* (2010) [21], is a partially ordered monoid $(P, \leq, \cdot, 1, -^l, -^r)$ where each object p has a left adjoint and a right adjoint, p^l and p^r respectively, where we have the following:

$$p^l \cdot p \leq 1 \leq p \cdot p^l, p \cdot p^r \leq 1 \leq p^r \cdot p.$$

The following reductions are axiomatic in a pregroup:

$$\epsilon^l_p = p^l \cdot p \leq 1,$$
$$\epsilon^r_p = p \cdot p^r \leq 1,$$
$$\eta^l_p = 1 \leq p \cdot p^l,$$
$$\eta^r_p = 1 \leq p^r \cdot p.$$

A pregroup grammar, developed by Lambek (2008) [24] is a pregroup that has a set of atomic grammar types. For example, given a pregroup grammar with types n and s to represent noun types and sentence types respectively, a sentence with a transitive verb such as "John cooks meals" with John: n, meals: n, and John cooks meals: s can be derived in the following way:

$$\begin{aligned} & n \cdot (n^r \cdot s \cdot n^l) \cdot n \\ & \quad \to (n \cdot n^r) \cdot s \cdot (n^l \cdot n) \\ & \quad \to (1) \cdot s \cdot (1) \\ & \quad \to s. \end{aligned}$$

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

The transitive verb type $n^r \cdot s \cdot n^l$ indicates that in order for the sentence to be grammatically valid of type s, there must be a phrase of type n to the left and right of the transitive verb. The sentence "John cooks meals" can also be represented with the string diagram in Fig. 10.

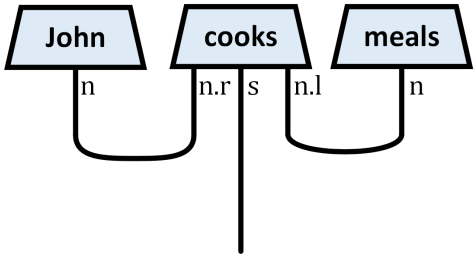


Fig. 10: String diagram of "John cooks meals"

The adjective type $n \cdot n^l$ indicates that a phrase composed of an adjective type and a noun phrase would create another noun phrase, as can be shown in the derivation of the sentence "John cooks tasty meals" below:

$$\begin{aligned} & n \cdot (n^r \cdot s \cdot n^l) \cdot (n \cdot n^l) \cdot n \\ & \quad \rightarrow (n \cdot n^r) \cdot s \cdot (n^l \cdot n) \cdot (n^l \cdot n) \\ & \quad \rightarrow (1) \cdot s \cdot (1) \cdot (1) \\ & \quad \rightarrow s. \end{aligned}$$

The sentence "John cooks tasty meals" can also be represented with the string diagram Fig. 11.

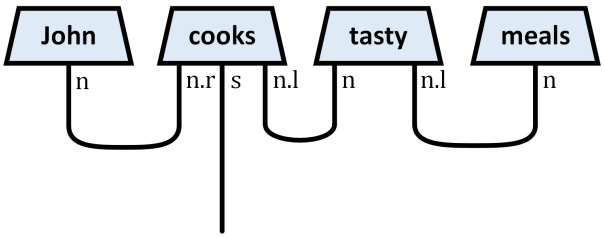


Fig. 11: String diagram of "John cooks tasty meals"

According to Sadrzadeh *et al.* (2013) [22], pregroup can be interpreted in vector space by mapping between the pregroup and FVect, where FVect is the category of finite dimensional vector spaces and linear maps. FVect is a compact closed category where the tensor product is symmetric. Given the pregroup $Preg$ and the functor $F : Preg \rightarrow FVect$, the basic pregroup grammar types in $Preg$ have vector space representations, for example:

$$F(n) = N, F(s) = S, F(1) = I$$

Since the tensor product is commutative, it follows that for each object V in $FVect$, $V^l \cong V^r \cong V^*$ where V^l, V^r, and V^* are the left adjoint, right adjoint, and dual space of V respectively. Therefore, we have the following:

$$F(x^l) = F(x^r) = F(x)^*.$$

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.